



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Actividad de seguimiento 2

Programación Orientada a Objetos

Estudiante

John Mario Mejia Arango

1017243551 CC

Docente

Walter Hugo Arboleda Mazo

Universidad Nacional de Colombia

Sede Medellín

2026 - 2S

08 de octubre de 2026

Este es el taller de seguimiento realizado por el estudiante John Mario Mejía Arango, cada punto fue realizado en google colab. El repositorio en Git hub lo encontrara en el siguiente enlace:

<https://github.com/jomejiaa/POO-Actividad-de-seguimiento-2-estudiante-John-Mario-Mejia-Arango.git>

Los ejercicios fueron realizados en Google Colab, fui un poco más hábil y se logró realizar los diagramas UML para cada ejercicio.

Ejercicio 2.1

<https://colab.research.google.com/drive/1K-JDHF4nrydczWX2fDshGykSvHFADR-#scrollTo=VFniicOZ1p1Q>

En el pasado link puede encontrar la solución en google colab, a continuación, se adjunta el código.

Código:

```
class Persona:
    def __init__(self, nombre: str, apellido: str, documento: int,
nacimiento: int):
        # Constructor que inicializa los atributos
        self.nombre = nombre
        self.apellido = apellido
        self.documento = documento
        self.nacimiento = nacimiento

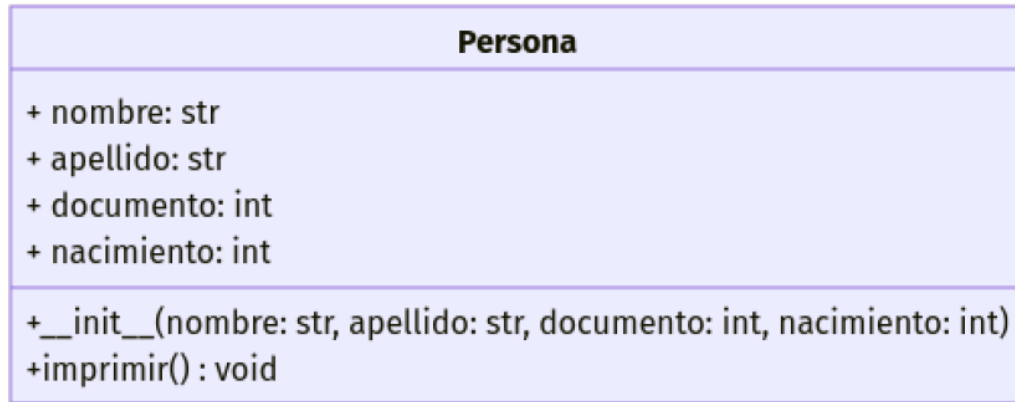
    def imprimir(self):
        # Método para mostrar los datos de la persona
        print(f"Nombre: {self.nombre}")
        print(f"Apellido: {self.apellido}")
        print(f"Documento: {self.documento}")
        print(f"Nacimiento: {self.nacimiento}")

def main():
    # Crear dos personas
    personal = Persona("Pedro", "Pérez", 1053121010, 1998)
    persona2 = Persona("Luis", "León", 105322344, 2001)

    # Imprimir los valores de los atributos de las personas
    print(" PERSONA #1")
    personal.imprimir()
    print() # Espacio entre las personas
    print(" PERSONA #2")
    persona2.imprimir()
```

```
# Llamada a la función main
if __name__ == "__main__":
    main()
```

Diagrama UML.



Ejercicio 2.2

https://colab.research.google.com/drive/1XJXyN1UNsOsa_79s6oNyD7H0ba5ZKM_Fv#scrollTo=FjT5LSZJ6fmw

En el pasado link puede encontrar la solución en google colab, a continuación, se adjunta el código.

Código:

```
from enum import Enum

class TipoPlaneta(Enum):
    GASEOSO = 1
    TERRESTRE = 2
    ENANO = 3

class Planeta:
    def __init__(
        self,
        nombre: str = None,
        num_satelites: int = 0,
        masa: float = 0.0,
        volumen: float = 0.0,
        diametro: int = 0,
        distancia_sol: int = 0,
        tipo: TipoPlaneta = None,
        observable: bool = False
    ):
        # Constructor que inicializa los atributos
```

```

        self.nombre = nombre
        self.num_satelites = num_satelites
        self.masa = masa
        self.volumen = volumen
        self.diametro = diametro
        self.distancia_sol = distancia_sol
        self.tipo = tipo
        self.observable = observable

def mostrar_info(self):
    # Método que imprime los valores de los atributos
    print(f"Nombre del planeta = {self.nombre}")
    print(f"Cantidad de satélites = {self.num_satelites}")
    print(f"Masa del planeta (kg) = {self.masa}")
    print(f"Volumen del planeta (km3) = {self.volumen}")
    print(f"Diámetro del planeta (km) = {self.diametro}")
    print(f"Distancia media al sol (km) = {self.distancia_sol}")
    print(f"Tipo de planeta = {self.tipo.name if self.tipo else
'No definido'}")
    print(f"Es observable a simple vista = {self.observable}")

def densidad(self):
    # Cálculo de la densidad
    if self.volumen > 0:
        return self.masa / self.volumen
    return 0

def es_exterior(self):
    UA = 149597870
    # Es exterior si está más allá de 3.4 UA
    return self.distancia_sol > 3.4 * UA

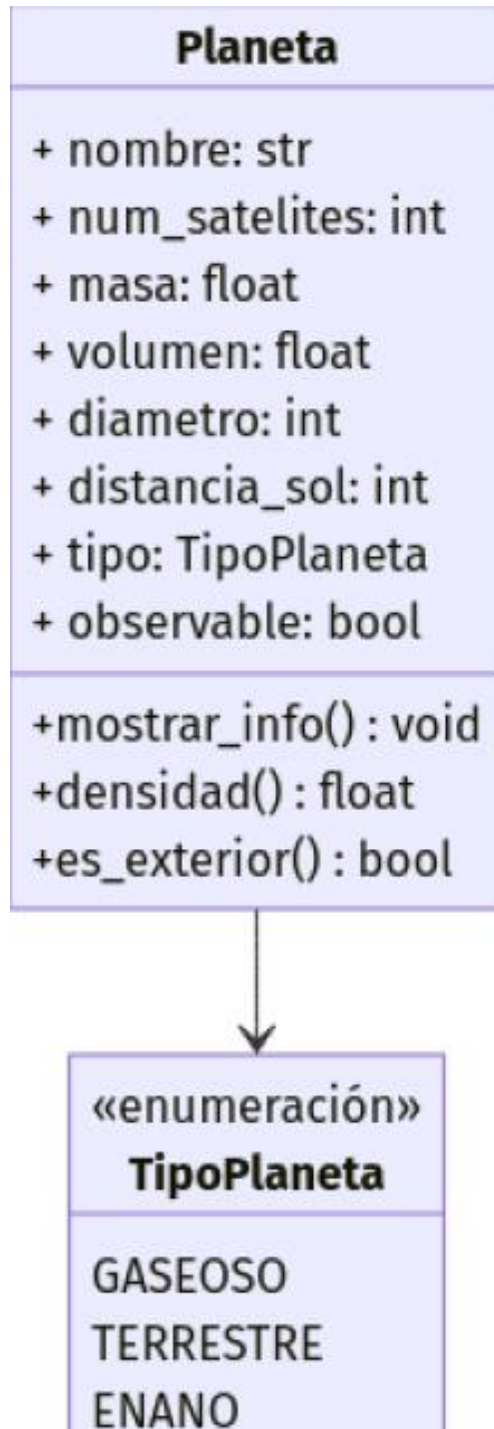
if __name__ == "__main__":
    # Crear dos planetas
    planeta1 = Planeta("Tierra", 1, 5.9736E24, 1.08321E12, 12742,
150000000, TipoPlaneta.TERRESTRE, True)
    planeta2 = Planeta("Júpiter", 79, 1.898E27, 1.4313E15, 139820,
778500000, TipoPlaneta.GASEOSO, True)

    # Mostrar info
    planeta1.mostrar_info()
    print(f"Densidad de {planeta1.nombre} = {planeta1.densidad()}
kg/km3")
    print(f"¿Es exterior? {planeta1.es_exterior()}\n")

```

```
planeta2.mostrar_info()
print(f"Densidad de {planeta2.nombre} = {planeta2.densidad()}
kg/km3")
print(f"¿Es exterior? {planeta2.es_exterior()}")
```

Diagrama UML.



Ejercicio 2.3

https://colab.research.google.com/drive/1xGYkMiY0Kd8V_yhh8vthp4i2fDgiRAV5#scrollTo=3nksO6vE81ON

En el pasado link puede encontrar la solución en google colab, a continuación, se adjunta el código.

Código:

```
from enum import Enum

class TipoCombustible(Enum):
    GASOLINA = 1
    BIOETANOL = 2
    DIESEL = 3
    BIODIESEL = 4
    GAS_NATURAL = 5

class TipoAutomovil(Enum):
    CARRO_CUIDAD = 1
    SUBCOMPACTO = 2
    COMPACTO = 3
    FAMILIAR = 4
    EJECUTIVO = 5
    SUV = 6

class TipoColor(Enum):
    BLANCO = 1
    NEGRO = 2
    ROJO = 3
    NARANJA = 4
    AMARILLO = 5
    VERDE = 6
    AZUL = 7
    VIOLETA = 8

class Automovil:
    def __init__(
        self,
        marca: str,
        modelo: int,
        motor: int,
        tipo_combustible: TipoCombustible,
        tipo_automovil: TipoAutomovil,
        numero_puertas: int,
        cantidad_asientos: int,
```

```

        velocidad_maxima: int,
        color: TipoColor
    ):
        self.marca = marca
        self.modelo = modelo
        self.motor = motor
        self.tipo_combustible = tipo_combustible
        self.tipo_automovil = tipo_automovil
        self.numero_puertas = numero_puertas
        self.cantidad_asientos = cantidad_asientos
        self.velocidad_maxima = velocidad_maxima
        self.color = color
        self.velocidad_actual = 0

# Lectura o tener el valor del atributo de un objeto
def get_marca(self):
    return self.marca

def get_modelo(self):
    return self.modelo

def get_motor(self):
    return self.motor

def get_tipo_combustible(self):
    return self.tipo_combustible

def get_tipo_automovil(self):
    return self.tipo_automovil

def get_numero_puertas(self):
    return self.numero_puertas

def get_cantidad_asientos(self):
    return self.cantidad_asientos

def get_velocidad_maxima(self):
    return self.velocidad_maxima

def get_color(self):
    return self.color

def get_velocidad_actual(self):
    return self.velocidad_actual

```

```

# Escritura o asignar o cambiar el valor de un atributo de un
objeto
def set_marca(self, marca):
    self.marca = marca

def set_modelo(self, modelo):
    self.modelo = modelo

def set_motor(self, motor):
    self.motor = motor

def set_tipo_combustible(self, tipo_combustible):
    self.tipo_combustible = tipo_combustible

def set_tipo_automovil(self, tipo_automovil):
    self.tipo_automovil = tipo_automovil

def set_numero_puertas(self, numero_puertas):
    self.numero_puertas = numero_puertas

def set_cantidad_asientos(self, cantidad_asientos):
    self.cantidad_asientos = cantidad_asientos

def set_velocidad_maxima(self, velocidad_maxima):
    self.velocidad_maxima = velocidad_maxima

def set_color(self, color):
    self.color = color

def set_velocidad_actual(self, velocidad_actual):
    self.velocidad_actual = velocidad_actual

def acelerar(self, aceleracion):
    if self.velocidad_actual + aceleracion >
self.velocidad_maxima:
        self.velocidad_actual = self.velocidad_maxima
        print("No se puede superar la velocidad maxima")
    else:
        self.velocidad_actual += aceleracion

def desacelerar(self, desaceleracion):
    if self.velocidad_actual - desaceleracion < 0:
        self.velocidad_actual = 0
        print("No se puede decrementar a una velocidad negativa.")
    else:

```

```

        self.velocidad_actual -= desaceleracion

def frenar(self):
    self.velocidad_actual = 0

def calcular_tiempo_llegada(self, distancia):
    if self.velocidad_actual == 0:
        return float('inf')
    return distancia / self.velocidad_actual

def imprimir(self):
    print(f"Marca = {self.marca}")
    print(f"Modelo = {self.modelo}")
    print(f"Motor = {self.motor}")
    print(f"Tipo de combustible = {self.tipo_combustible.name}")
    print(f"Tipo de automovil = {self.tipo_automovil.name}")
    print(f"Numero de puertas = {self.numero_puertas}")
    print(f"Cantidad de asientos = {self.cantidad_asientos}")
    print(f"Velocidad maxima = {self.velocidad_maxima}")
    print(f"Color = {self.color.name}")

def imprimir_velocidad_actual(self):
    print(f"Velocidad actual = {self.velocidad_actual}")

class Prueba_Automovil:
    @staticmethod
    def main():
        # Objeto
        auto1 = Automovil(
            "Ford",
            2018,
            3,
            TipoCombustible.DIESEL,
            TipoAutomovil.EJECUTIVO,
            5,
            6,
            250,
            TipoColor.NEGRO
        )

        auto1.imprimir()
        auto1.set_velocidad_actual(100)
        auto1.imprimir_velocidad_actual()

        auto1.acelerar(20)

```

```

        auto1.imprimir_velocidad_actual()

        auto1.desacelerar(50)
        auto1.imprimir_velocidad_actual()

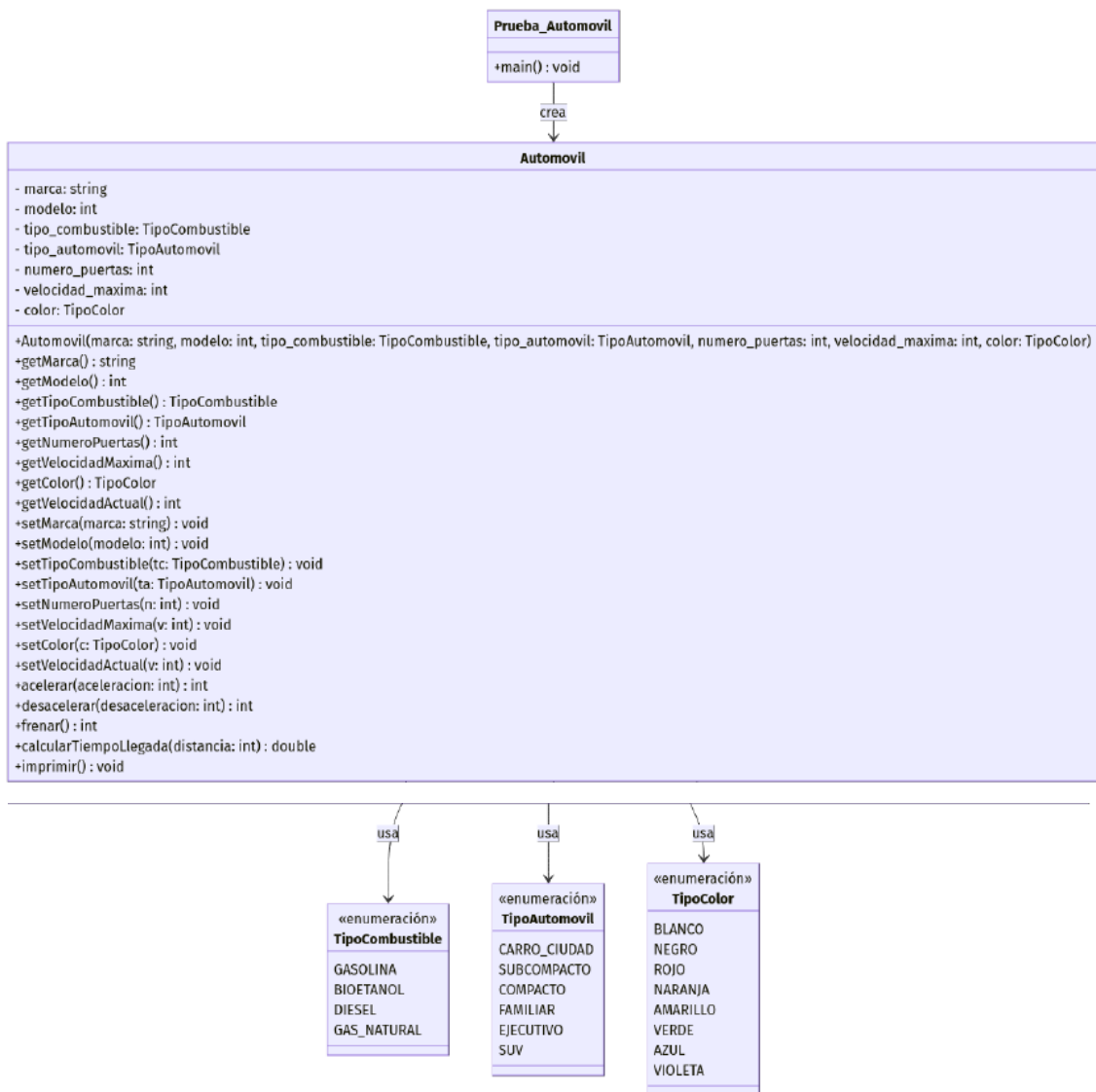
        auto1.frenar()
        auto1.imprimir_velocidad_actual()

        auto1.desacelerar(20)

if __name__ == "__main__":
    Prueba_Automovil.main()

```

Diagrama UML.



Ejercicio 2.4

https://colab.research.google.com/drive/1i3ljn7eCV8oQjRrO7tCbyY5_NbMA5MI

En el pasado link puede encontrar la solución en google colab, a continuación, se adjunta el código.

Código:

```
import math

class Circulo:
    def __init__(self, radio):
        self.radio = radio

    def calcular_area(self):
        return math.pi * self.radio ** 2

    def calcular_perimetro(self):
        return 2 * math.pi * self.radio

class Rectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return self.base * self.altura

    def calcular_perimetro(self):
        return 2 * (self.base + self.altura)

class Cuadrado:
    def __init__(self, longitud):
        self.longitud = longitud

    def calcular_area(self):
        return self.longitud ** 2

    def calcular_perimetro(self):
        return 4 * self.longitud

class TrianguloRectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura
```

```

def calcular_area(self):
    return (self.base * self.altura) / 2

def calcular_hipotenusa(self):
    return math.sqrt(self.base ** 2 + self.altura ** 2)

def calcular_perimetro(self):
    return self.base + self.altura + self.calcular_hipotenusa()

def determinar_tipo_triagunlo(self):
    hipotenusa = self.calcular_hipotenusa()
    if self.base == self.altura and self.base == hipotenusa and
self.altura == hipotenusa:
        return "Es un triangulo Equilatero"
    elif self.base != self.altura and self.base != hipotenusa and
self.altura != hipotenusa:
        return "Es un triangulo Escaleno"
    else:
        return "Es un triangulo Isosceles"

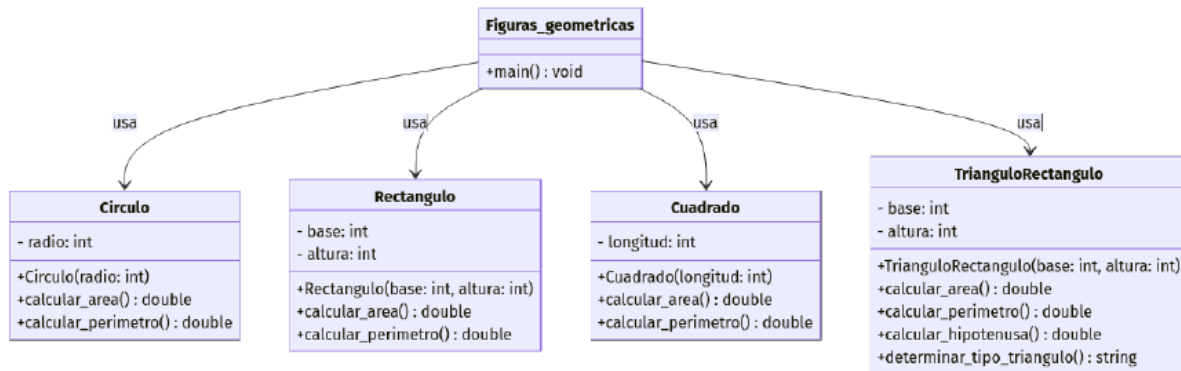
# Clase principal
class Figuras_geometricas:
    @staticmethod
    def main():
        figura1 = Circulo(2.0)
        figura2 = Rectangulo(1, 2)
        figura3 = Cuadrado(3)
        figura4 = TrianguloRectangulo(3, 5)

        print(f"El area del circulo es = {figura1.calcular_area()}")
        print(f"El perimetro del circulo es =
{figura1.calcular_perimetro()}")
        print(f"El area del rectangulo es =
{figura2.calcular_area()}")
        print(f"El perimetro del rectangulo es =
{figura2.calcular_perimetro()}")
        print(f"El area del cuadrado es = {figura3.calcular_area()}")
        print(f"El perimetro del cuadrado es =
{figura3.calcular_perimetro()}")
        print(f"El area del triangulo rectangulo es =
{figura4.calcular_area()}")
        print(f"El perimetro del triangulo rectangulo es =
{figura4.calcular_perimetro()}")
        print(figura4.determinar_tipo_triagunlo())

```

```
if __name__ == "__main__":
    Figuras_geometricas.main()
```

Diagrama UML.



Ejercicio 2.5

https://colab.research.google.com/drive/1aEaaJ9uko9ycaJRqF2Cq7Sbs0hy_uCF4

En el pasado link puede encontrar la solución en google colab, a continuación, se adjunta el código.

Código:

```
class CuentaBancaria:
    def __init__(self, nombresTitular, apellidosTitular, numeroCuenta,
tipoCuenta):
        self.nombresTitular = nombresTitular
        self.apellidosTitular = apellidosTitular
        self.numeroCuenta = numeroCuenta
        self.tipoCuenta = tipoCuenta
        self.saldo = 0.0

    def imprimir(self):
        print("Nombres del titular =", self.nombresTitular)
        print("Apellidos del titular =", self.apellidosTitular)
        print("Número de cuenta =", self.numeroCuenta)
        print("Tipo de cuenta =", self.tipoCuenta)
        print("Saldo =", self.saldo)

    def consultarSaldo(self):
        print("El saldo actual es =", self.saldo)

    def consignar(self, valor):
        if valor > 0:
            self.saldo += valor
```

```

        print(f"Se ha consignado ${valor}. Nuevo saldo:
${self.saldo}")
        return True
    print("El valor a consignar debe ser mayor que cero.")
    return False

    def retirar(self, valor):
        if valor > 0 and valor <= self.saldo:
            self.saldo -= valor
            print(f"Se ha retirado ${valor}. Nuevo saldo:
${self.saldo}")
            return True
        print("El valor a retirar debe ser menor o igual al saldo
actual y mayor que cero.")
        return False

    def compararCuenta(self, cuenta):
        return (self.numeroCuenta == cuenta.numeroCuenta
                and self.nombresTitular == cuenta.nombresTitular
                and self.apellidosTitular == cuenta.apellidosTitular
                and self.tipoCuenta == cuenta.tipoCuenta)

    def transferencia(self, cuenta, valor):
        if self.retirar(valor):
            cuenta.consignar(valor)
            print(f"Se ha transferido ${valor} de la cuenta
{self.numeroCuenta} a {cuenta.numeroCuenta}")
            return True
        print("No se pudo realizar la transferencia.")
        return False

if __name__ == "__main__":
    cuental = CuentaBancaria("Pedro", "Pérez", 123456, "AHORROS")
    cuenta2 = CuentaBancaria("Juan", "López", 654321, "CORRIENTE")
    cuental.imprimir()
    cuental.consignar(500000)
    cuental.transferencia(cuenta2, 200000)
    cuental.consultarSaldo()
    cuenta2.consultarSaldo()

```

Diagrama UML.

